

Лабораторная работа № 13

Мультиагентные системы: разработка распределённых интеллектуальных агентов

Цель: освоить проектирование и реализацию мультиагентных систем (MAS), где агенты решают задачи из конкретной предметной области, взаимодействуют через брокеры сообщений, а оркестратор управляет их работой.

Теоретические положения:

Изучить:

- **Мультиагентные системы (MAS)** — парадигма, в которой несколько автономных агентов взаимодействуют для достижения общей или индивидуальных целей.
- **Типы агентов:** реактивные (реагируют на события), целевые (стремятся к достижению цели), BDI-агенты (Belief-Desire-Intention).
- **Коммуникация агентов:** через брокеры сообщений (NATS, RabbitMQ, MQTT), очереди, каналы.
- **Оркестрация:** централизованный управляющий компонент (на Python) распределяет задачи между агентами.
- **Фреймворки:** LangChain (для LLM-агентов), AutoGen (Microsoft), CrewAI, CAMEL.
- **Шаблоны взаимодействия:** цепочка (chain), конвейер (pipeline), чёрная доска (blackboard), аукцион (auction).

Ссылки:

- NATS: <https://nats.io/>
- RabbitMQ: <https://www.rabbitmq.com/>
- LangChain: <https://www.langchain.com/>
- AutoGen: <https://microsoft.github.io/autogen/>
- CrewAI: <https://www.crewai.com/>

Пример агента на Go (обработчик задач):

package main

```
import (  
    "encoding/json"  
    "log"  
    "github.com/nats-io/nats.go"  
)  
  
type Task struct {  
    ID    string `json:"id"`  
    Type  string `json:"type"`  
    Payload string `json:"payload"`  
}  
  
type Result struct {  
    TaskID string `json:"task_id"`  
    Success bool  `json:"success"`  
}
```

```

    Output string `json:"output"`
}

func main() {
    nc, _ := nats.Connect(nats.DefaultURL)
    defer nc.Close()

    nc.Subscribe("tasks.process", func(m *nats.Msg) {
        var task Task
        json.Unmarshal(m.Data, &task)

        log.Printf("Агент получил задачу %s типа %s", task.ID, task.Type)

        // Обработка в зависимости от типа задачи
        output := processTask(task)

        result := Result{TaskID: task.ID, Success: true, Output: output}
        response, _ := json.Marshal(result)
        nc.Publish("tasks.completed", response)
    })

    select {}
}

func processTask(task Task) string {
    return "Обработано: " + task.Payload
}

```

Пример оркестратора на Python:

```

import asyncio
import json
import uuid
import nats
from typing import Dict, Optional

class AgentOrchestrator:
    def __init__(self):
        self.nc: Optional[nats.NATS] = None
        self.results: Dict[str, asyncio.Future] = {}
        self.subscription = None

    async def connect(self):
        self.nc = await nats.connect("nats://localhost:4222")

    async def send_task(self, task_type: str, payload: dict, timeout: int = 30) -> dict:
        task_id = str(uuid.uuid4())
        task = {
            "id": task_id,
            "type": task_type,
            "payload": json.dumps(payload)
        }

```

```

# Ожидание результата
future = asyncio.Future()
self.results[task_id] = future

# Отправка задачи
await self.nc.publish("tasks.process", json.dumps(task).encode())

# Ожидание с таймаутом
try:
    result = await asyncio.wait_for(future, timeout)
    return json.loads(result)
except asyncio.TimeoutError:
    del self.results[task_id]
    raise TimeoutError(f"Задача {task_id} не выполнена за {timeout} сек")

async def on_result(self, msg):
    result = json.loads(msg.data.decode())
    if result["task_id"] in self.results:
        self.results[result["task_id"]].set_result(msg.data.decode())
        del self.results[result["task_id"]]

async def start_listener(self):
    await self.nc.subscribe("tasks.completed", cb=self.on_result)
    # Бесконечное ожидание или run_forever()

```

ПРАКТИЧЕСКИЕ ЗАДАНИЯ

Индивидуальные варианты по предметным областям:

Ниже представлены **30 индивидуальных вариантов**, каждый из которых задаёт конкретную **предметную область** для разработки мультиагентной системы. Для каждой области определён **набор агентов** (3–5 типов) и **сценарии взаимодействия**.

Вариант	Предметная область	Типы агентов	Сложность
1	Служба доставки еды	Приём заказов, Маршрутизация курьеров, Управление ресторанами, Уведомления	Средний
2	Автоматизация техподдержки	Классификация тикетов, Поиск по базе знаний, Генерация ответов, Эскалация	Средний
3	Управление складом	Приёмка товара, Размещение на полках, Комплектация заказов, Инвентаризация	Средний

Вариант	Предметная область	Типы агентов	Сложность
4	Система рекомендаций	Сбор данных о пользователе, Поиск похожих пользователей, Генерация рекомендаций, Оценка релевантности	Средний
5	Мониторинг IT-инфраструктуры	Сбор метрик, Детекция аномалий, Оповещение, Автоматическое восстановление	Средний
6	Бронирование гостиниц	Поиск свободных номеров, Бронирование, Обработка оплаты, Управление отменами	Средний
7	Торговая платформа	Сбор котировок, Анализ рынка, Исполнение сделок, Управление рисками	Средний
8	Автоматизация HR	Парсинг резюме, Сопоставление с вакансиями, Планирование собеседований, Обратная связь	Средний
9	Умный дом	Управление освещением, Климат-контроль, Безопасность, Энергоменеджмент	Средний
10	Обработка заказов в ресторане	Приём заказа, Кухня, Управление столами, Доставка	Средний
11	Система логистики магазина	Планирование маршрутов, Управление ТС, Обработка заказов, Отслеживание грузов	Повышенный
12	Медицинская диагностика	Сбор симптомов, Анализ истории болезней, Постановка предварительного диагноза, Назначение исследований	Повышенный
13	Анализ социальных сетей	Сбор постов, Анализ тональности, Выявление трендов, Генерация отчётов	Повышенный
14	Управление проектами	Распределение задач, Контроль сроков, Управление ресурсами, Бюджетирование	Повышенный

Вариант	Предметная область	Типы агентов	Сложность
15	Автоматизация маркетинга	Сегментация аудитории, Создание рассылок, Анализ отклика, Оптимизация кампаний	Повышенный
16	Управление производством	Планирование загрузки, Контроль качества, Управление запасами, Диспетчеризация	Повышенный
17	Кибербезопасность (SIEM)	Сбор логов, Корреляция событий, Детекция атак, Блокировка трафика	Повышенный
18	Система поддержки пациентов	Запись к врачу, Напоминание о приёме, Сбор обратной связи, Тriage	Повышенный
19	Финтех: кредитный скоринг	Сбор данных о клиенте, Анализ доходов, Оценка риска, Формирование решения	Повышенный
20	Управление парковкой	Поиск свободных мест, Бронирование, Контроль доступа, Расчёт оплаты	Повышенный
21	Автоматизация юридической консультации	Анализ запроса, Поиск по документам, Формирование ответа, Проверка на противоречия	Повышенный
22	Управление автопарком	Планирование ТО, Мониторинг топлива, Диспетчеризация, Анализ эффективности	Повышенный
23	Система e-learning	Рекомендация курсов, Проверка заданий, Анализ прогресса, Генерация сертификатов	Повышенный
24	Анализ рынка недвижимости	Сбор объявлений, Анализ цен, Прогнозирование спроса, Оценка недвижимости	Повышенный
25	Управление цепочками поставок	Прогнозирование спроса, Заказ у поставщиков, Отслеживание поставок, Управление рисками	Повышенный

Вариант	Предметная область	Типы агентов	Сложность
26	Система борьбы с мошенничеством	Сбор транзакций, Анализ паттернов, Оценка риска, Блокировка	Повышенный
27	Автоматизация тестирования ПО	Генерация тестов, Запуск тестов, Анализ покрытия, Создание отчётов	Повышенный
28	Управление персоналом	Планирование смен, Отслеживание отпусков, Кадровый учёт, Расчёт зарплаты	Повышенный
29	Система управления гостиницей	Заселение/выселение, Уборка комнат, Обработка запросов гостей, Билинг	Повышенный
30	Анализ энергопотребления	Сбор показаний, Детекция аномалий, Прогнозирование нагрузки, Оптимизация	Повышенный

**Общий список заданий
(выполняются в рамках своей предметной области)**

Задания средней сложности (варианты 1–10):

1. Определение агентов и их ролей

Для своей предметной области выделить типы агентов (не менее 3) и описать их: роль, входные данные, выходные данные, бизнес-правила.

2. Разработка прототипа агента на Go

Реализовать одного из агентов на Go, который подписывается на очередь NATS и обрабатывает задачи своего типа. Агент должен принимать структурированные данные (JSON), выполнять простую логику обработки и публиковать результат.

3. Разработка оркестратора на Python

Создать оркестратор на Python с использованием библиотеки asyncio и nats-пу. Оркестратор должен уметь отправлять задачи агенту, ожидать результаты и обрабатывать таймауты.

4. Настройка коммуникации через NATS

Развернуть NATS в Docker (docker-compose). Настроить взаимодействие между оркестратором и агентом, убедиться, что сообщения доставляются корректно.

5. Логирование и мониторинг

Добавить логирование в оркестратор и агента (уровни INFO, ERROR). Организовать вывод логов в консоль и файл. Добавить простую метрику (счётчик обработанных задач).

6. Обработка ошибок и таймаутов

Реализовать в оркестраторе механизм повторной отправки задачи при сбое агента (retry, не более 3 раз). Добавить таймаут ожидания результата.

7. Запуск нескольких агентов одного типа

Запустить 2–3 экземпляра одного и того же Go-агента. Продемонстрировать балансировку нагрузки: оркестратор может отправлять задачи, и они распределяются между агентами.

8. Создание API для запуска задач

Разработать простой REST API на FastAPI (Python), который принимает запросы на выполнение задач (HTTP → оркестратор → агенты → ответ).

9. Тестирование системы

Написать модульные тесты для агента на Go (используя стандартный пакет testing) и для оркестратора на Python (pytest с моками NATS).

10. Документирование архитектуры

Создать диаграмму взаимодействия агентов (PlantUML или Mermaid) и описание каждого компонента системы.

Задания повышенной сложности (варианты 11–30):

1. Разработка полной системы из 3–5 агентов на Go

Реализовать всех агентов, определённых для своей предметной области. Каждый агент — отдельный микросервис на Go, общающийся через NATS. Агенты могут быть разных типов (например, анализатор, валидатор, исполнитель).

2. Цепочки задач (pipeline)

Реализовать последовательную обработку задачи несколькими агентами. Например: Агент А принимает задачу → отправляет результат в очередь для Агента В → Агент В обрабатывает → и т.д. Оркестратор управляет цепочкой.

3. Распределённая трассировка (Jaeger)

Интегрировать OpenTelemetry в агенты и оркестратор. Настроить сбор трассировок в Jaeger (запущенном в Docker). Визуализировать полный путь выполнения распределённой задачи.

4. Агент с состоянием (Redis)

Реализовать агента, который сохраняет своё состояние в Redis (например, счётчики, статистику или кэш результатов). При перезапуске агент восстанавливает состояние.

5. Динамическое масштабирование

Реализовать систему, которая автоматически запускает дополнительные экземпляры агента при высокой нагрузке (например, при длине очереди > N). Использовать Docker API или оркестратор (Kubernetes).

6. Аукционное распределение задач

Реализовать модель, где агенты «торгуются» за задачу: каждый агент оценивает свою способность выполнить задачу (cost, skill, availability). Оркестратор выбирает агента с наименьшей стоимостью или наилучшим соответствием.

7. Интеграция LLM-агента

Добавить в систему агента на Python, использующего LLM (локальную через Ollama или облачную через API). Агент должен выполнять интеллектуальную задачу в рамках предметной области (например, генерация ответа, классификация текста).

8. Веб-интерфейс для мониторинга агентов

Разработать веб-панель (Streamlit или FastAPI + Jinja2) для отображения статуса агентов, очередей, результатов задач, возможности ручного запуска задач.

Пример адаптации задания под предметную область

Вариант 1 (средний уровень): Служба доставки еды

Агенты:

Агент	Роль	Вход	Выход
Приём заказов	Принимает заказы от API, валидирует	JSON-заказ	Валидированный заказ
Маршрутизация курьеров	Назначает курьера на заказ	Заказ, местоположение курьеров	Назначенный курьер
Управление ресторанами	Проверяет доступность блюд	Заказ, ресторан	Готовность к приготовлению
Уведомления	Отправляет статусы клиенту	Статус заказа	SMS/email уведомление

Задание 1 (среднее): Описать роли агентов, входные/выходные данные и бизнес-правила для каждого.

Задание 6 (среднее): Реализовать обработку ошибок: если ресторан не подтвердил готовность за 5 минут, оркестратор повторяет запрос другому ресторану (retry, fallback).

Задание 2 (повышенное, для варианта 11+): Цепочка: Приём заказа → Проверка в ресторане → Назначение курьера → Уведомления. Реализовать pipeline, где результат одного агента идёт на вход следующему.

Индивидуальные варианты по предметным областям

Вариант	Предметная область	Типы агентов (не менее 3)
1	Служба доставки еды	Приём заказов, Маршрутизация курьеров, Управление ресторанами, Уведомления
2	Автоматизация техподдержки	Классификация тикетов, Поиск по БЗ, Генерация ответов, Эскалация
3	Управление складом	Приёмка, Размещение, Комплектация, Инвентаризация
4	Система рекомендаций	Сбор данных, Поиск похожих, Генерация рекомендаций, Оценка
5	Мониторинг IT-инфраструктуры	Сбор метрик, Детекция аномалий, Оповещение, Восстановление

Вариант	Предметная область	Типы агентов (не менее 3)
6	Бронирование гостиниц	Поиск номеров, Бронирование, Оплата, Отмена
7	Торговая платформа	Котировки, Анализ, Исполнение сделок, Риски
8	Автоматизация HR	Парсинг резюме, Сопоставление, Планирование собеседований, Обратная связь
9	Умный дом	Освещение, Климат, Безопасность, Энергоменеджмент
10	Обработка заказов в ресторане	Приём заказа, Кухня, Столы, Доставка
11	Логистика магазина	Маршруты, ТС, Заказы, Трекинг
12	Медицинская диагностика	Симптомы, История, Диагноз, Исследования
13	Анализ соцсетей	Сбор, Тональность, Тренды, Отчёты
14	Управление проектами	Задачи, Сроки, Ресурсы, Бюджет
15	Автоматизация маркетинга	Сегментация, Рассылки, Аналитика, Оптимизация
16	Управление производством	Загрузка, Качество, Запасы, Диспетчеризация
17	Кибербезопасность (SIEM)	Логи, Корреляция, Детекция, Блокировка
18	Поддержка пациентов	Запись, Напоминания, Обратная связь, Тriage
19	Кредитный скоринг	Данные клиента, Доходы, Риск, Решение
20	Управление парковкой	Поиск мест, Бронирование, Доступ, Оплата
21	Юридическая консультация	Анализ запроса, Поиск документов, Ответ, Проверка
22	Управление автопарком	ТО, Топливо, Диспетчеризация, Аналитика

Вариант	Предметная область	Типы агентов (не менее 3)
23	E-learning	Рекомендации, Проверка, Прогресс, Сертификаты
24	Анализ рынка недвижимости	Сбор, Цены, Прогноз, Оценка
25	Управление цепочками поставок	Прогноз, Заказ, Трекинг, Риски
26	Борьба с мошенничеством	Транзакции, Паттерны, Риск, Блокировка
27	Автоматизация тестирования ПО	Генерация тестов, Запуск, Покрытие, Отчёты
28	Управление персоналом	Смены, Отпуска, Учёт, Зарплата
29	Управление гостиницей	Заселение, Уборка, Запросы, Билинг
30	Анализ энергопотребления	Показания, Аномалии, Прогноз, Оптимизация

Рекомендации к работе

1. **Выбор варианта** осуществляется преподавателем или по номеру студента. Варианты 1–10 (средняя сложность) выполняют 10 заданий из первого блока. Варианты 11–30 (повышенная сложность) выполняют 8 заданий из второго блока (все, кроме, возможно, самого сложного на усмотрение преподавателя).

2. **Технологический стек:** для агентов на Go требуется установленный Go, библиотека `nats.go`. Для оркестратора на Python — `nats-py`, `asyncio`. Для развёртывания NATS — Docker.

3. **Инфраструктура:** все компоненты (оркестратор, агенты, NATS) поднимаются локально с помощью Docker Compose. Это позволяет имитировать распределённую систему на одной машине.